Half of Your CMDB's Measured Value Is the Routing Queue You Already Have

Sudhir Dixit

Independent Researcher
sudhir.dixit1@gmail.com

Abstract

Configuration management database (CMDB) programmes are justified by the analytics they enable, and that justification is a comparison against a baseline the analyst chooses. On a public event log of $45{,}455$ incidents from a bank's IT operation whose affected-item field is 100% populated, we show the choice is worth as much as the data. Predicting at intake whether an incident will be reassigned, knowing the affected configuration item is worth $+0.183$ AUC against four intake fields, and $+0.103$ $[+0.094, +0.114]$ once the opening assignment queue is added — a field every service desk records at intake, for free. The gap survives three estimator families, including one in which item identity is a single column rather than $2{,}554$. Stated as a detection quantity it is far larger: at a fixed review capacity of 10% of arrivals, the queue-free baseline attributes 12.7 times as many additional catches to the CMDB as the queue-aware one does. Two measurements explain it. Adding the queue to a model that already knows the item is worth under 0.01 AUC; and the shrinkage is graded in the queue's resolution, rising from 27% when the queue is reduced to one bit to 44% at full resolution. The same shrinkage reproduces on two further targets and at three target thresholds. We report two attempts that failed — a third free-looking field drives the measured gain to zero and we could not establish whether it is admissible, and a reverse-direction measurement whose defensible nulls disagree.

1 Introduction

A CMDB programme is expensive and is justified by what it will enable. That justification rests on a comparison whose second arm — performance without configuration data — is a choice the analyst makes, usually silently, by deciding which already-available fields to include.

On one organisation's incident log the choice halves the answer, and we can say exactly where the difference goes. This is a short paper with one result and one mechanism.

What kind of contribution this is. We report no deployed system. This is a retrospective study on a public benchmark, and it belongs to this track as an evaluation practice: the quantity we measure is not a model's accuracy but the number a deployment decision turns on, and we show that number is set by a field-admission choice that is rarely written down. The organisations that make this choice are making it now, on data of exactly this shape, and Section 6 states what it costs them in the units a service desk actually operates in.

On what this paper used to claim. Earlier versions of this work reported several larger findings that were withdrawn under adversarial review, each an artifact of a control we had constructed. The paper is deliberately smaller as a result: every claim below is either a direct measurement or is reported against a null, and where a null is ambiguous we say so and drop the claim.

2 Background

The CMDB and what justifies it. A CMDB records configuration items and their relationships, and is the asset on which the configuration-management practice of ITIL and its successors rests. Survey work on ITSM framework adoption (Marrone and Kolbe 2011) finds that realised benefits grow with implementation maturity, but measures them as practitioner-reported outcomes rather than as the analytic value of the configuration data itself. We are not aware of a peer-reviewed measurement of what knowing the affected configuration item is worth to a named prediction task; the material addressing that question directly is vendor documentation. The gap is the occasion for this paper, and part of the reason it exists is that the data to close it barely does: of the three public ITSM incident logs in common use, one records the affected item on every incident, the second records it on 0.2%, and the third has no such field at all (van Dongen 2014; Amaral, Fantinato, and Peres 2018; Steeman 2013). That is also why this study is single-organisation, and it is a constraint rather than a choice.

The methods a CMDB is bought to enable. Surveys of AIOps for failure management (Zhang et al. 2024; Remil et al. 2024; Notaro, Cardoso, and Gerndt 2021) catalogue triage, failure-prediction and root-cause methods that take configuration data as input. Incident triage in particular — deciding which team owns an incident — has been treated as a learning problem on operational logs at industrial scale (Chen et al. 2019), as has failure prediction across a cloud estate (Lin et al. 2018).

Prediction on incident event logs. Our task sits in predictive process monitoring, where outcome prediction is sur-

veyed and benchmarked across real-world logs (Teinemaa et al. 2019; Verenich et al. 2019) on data of the kind process mining formalises (van der Aalst 2016). The BPI Challenge logs are the standard public instances (van Dongen 2014; Steeman 2013) and comparable ITSM exports exist elsewhere (Amaral, Fantinato, and Peres 2018). Such logs carry documented quality hazards: the imperfection patterns catalogued by Suriadi et al. (2017) include several we rule out explicitly below, in particular values denormalised from a record onto its events.

What the comparison is measured against. The quantity we report — the value of a feature *given* a baseline — is the object that algorithm-agnostic variable-importance frameworks formalise (Williamson et al. 2021), and the overlap we measure is exactly the feature interaction that additive importance measures are built to handle (Covert, Lundberg, and Lee 2020). That a feature's measured importance depends on what else the model already has is therefore not news as a statistical fact. What is not established is its magnitude for configuration data on a named operational task, or that the field it overlaps with is one the organisation already records for nothing. Our question is not whether AIOps methods work but what their evaluation is measured against. Rendle, Zhang, and Koren (2019) show for recommender systems that inadequately specified baselines can account for reported gains, and Ferrari Dacrema, Cremonesi, and Jannach (2019) report the same pattern across a body of neural recommendation work. Benchmark conclusions are sensitive to sources of variation that are seldom reported (Bouthillier et al. 2021), and documentation of the choices producing a result is thin across AI research (Gundersen and Kjensmo 2018). Where the choice concerns which fields may enter a model it shades into leakage, whose formulation and detection are treated by Kaufman et al. (2012) and whose consequences across empirical fields are catalogued by Kapoor and Narayanan (2023); for predictive process monitoring specifically, Weytjens and De Weerdt (2022) show that benchmark datasets need explicit construction to avoid it. We report this failure mode for a deployment decision rather than a leaderboard, where the field-admission choice is not a modelling detail but the number the business case turns on.

3 Data and Task

We use the incident records of the BPI Challenge 2014 log (van Dongen 2014) from Rabobank Group ICT, recorded in HP Service Manager, with the activity log shipped alongside.

Cleaning. Of $46{,}809$ rows, 203 carry no incident identifier and one an unparseable timestamp. A further $1{,}150$ incidents opened before 2013-10-01 are left-censored — already open when the extract began — and show reassignment rates of 76–100% against a subsequent 35–42% that declines gently across the window. Removing them leaves $\mathbf{45{,}455}$ incidents to 2014-03-31; a temporal $70/30$ split gives $31{,}818$ training and $13{,}637$ test incidents, positive rates 0.413 and 0.372. The cutoff is not taken from the outcome: monthly volume rises from 857 incidents in September 2013 to $8{,}606$ in October. Over the analysed rows the affected-item field is 100% populated across $2{,}929$ items, $2{,}554$ of them seen in training.

Baseline	AUC	+ item id.	gain [95% CI]
intake only	0.562	0.746	$+0.183$ [$+0.172$, $+0.195$]
+ routing queue	0.644	0.748	$+0.103$ [$+0.094$, $+0.114$]

Table 1: The value of knowing the affected configuration item, with and without a field the service desk already records.

Task and estimator. Predict at creation whether an incident will later be reassigned. We call the target reassignment rather than misrouting throughout: it fires on 37.2% of test incidents, which is routine handling and not error, and Section 6 reports the ladder at stricter thresholds. Median handling time rises from 1.59 hours for incidents never reassigned to 47.53 hours for those reassigned five or more times; we use this to motivate the task, not as a recoverable saving. The primary estimator is one-hot logistic regression, chosen because it is bin-free; two further estimator families are reported in Section 4. Encoders and rankings are fit on training data only; intervals are $2{,}000$-resample paired bootstraps.

The opening routing queue. The activity log records, on the `Open` event of every incident, the queue the service desk routed it to: 49 groups in training, none missing. The `Open` activity is the earliest activity for all $45{,}455$ incidents, its timestamp never precedes the record's open time, and the value varies within an incident for 92.56% of cases in the cohort — so it is a genuine per-event observation available at creation, not a closing state. It is also not the resolving queue: it matches the last-observed queue for only 21.35% of incidents. We note that an early version of this work excluded assignment group outright as a handling-time field. That ruling was about the queue recorded at closure; the value on the opening row is a different variable, and the checks above are why we admit it.

The queue is one pool plus a tail, and it drifts. Nominal cardinality overstates this field badly and we disclose its shape because it bears on how the result transfers. In training its 49 groups carry 2.43 bits, an effective cardinality of 5.4, with the largest group holding 62.1% of incidents; in test the live groups fall to 32 and the largest holds 78.6%. Most of what the queue knows is one contrast: the reassignment rate is 0.309 inside the dominant pool and 0.603 outside it. Section 5 shows the consequence, which is that the effect does not need the full field.

Field mutation, disclosed. Because the incident file is a closed-record snapshot, we report how often each field we use is edited after creation: Urgency on $1{,}107$ incidents (2.44%), Impact on $1{,}084$ (2.38%), the affected item on 159 (0.35%). Incidents whose Impact or Urgency was edited are more likely to be reassigned (0.66 against 0.39), so this is not neutral; Section 4 reports the headline with those fields dropped and with the edited incidents removed.

4 The Result

Against four intake fields the affected item looks decisive. Adding the opening queue cuts its measured value by 44%.

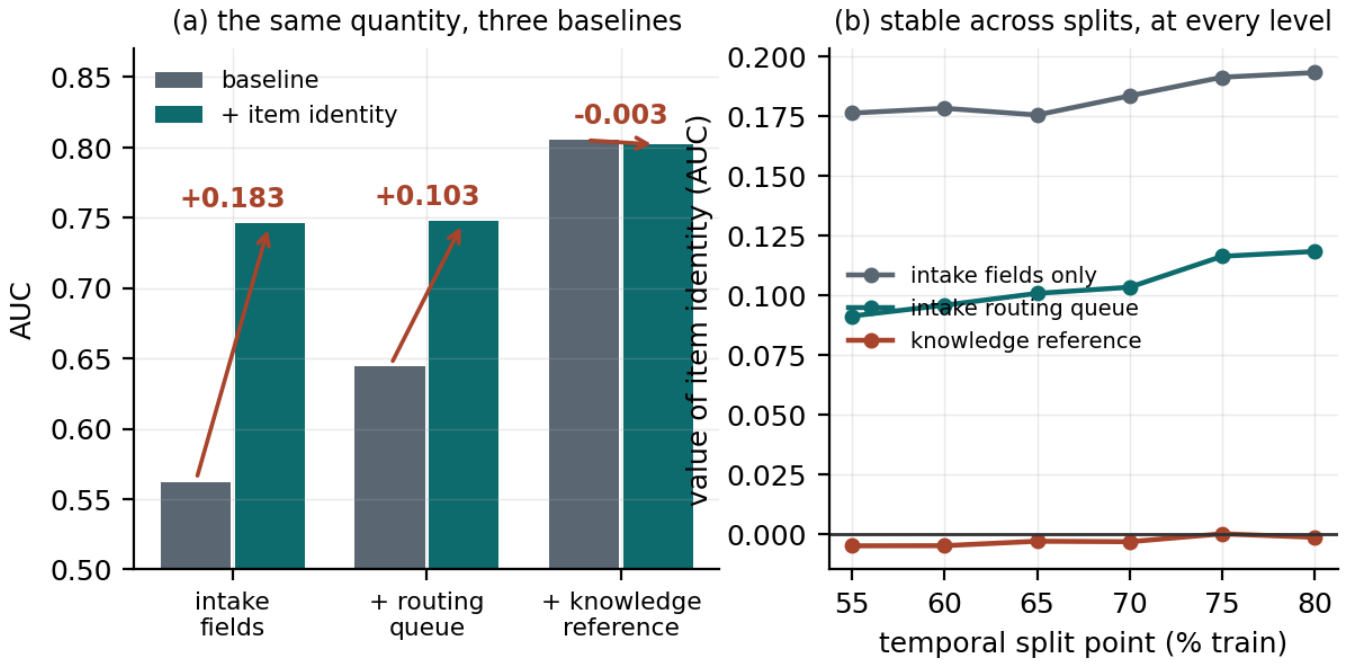

Figure 1: (a) The same quantity under three baselines; the third is discussed in Section 8. (b) The ordering holds at every temporal split point from 55% to 80%.

The gains are 28.1 and 17.4 standard deviations outside a null in which the incident-to-item association is shuffled, preserving column count and marginal distribution while destroying the signal — a comparison that matters because adding item identity adds 2,554 indicator columns, which is not free under a fixed penalty. These z figures pool the null's Monte-Carlo dispersion with the gain's own bootstrap standard error; dividing by the null alone, as an earlier draft did, would report 51 and 33.

The result is not a split artifact: across six temporal split points the two gains range $+0.176$ to $+0.193$ and $+0.091$ to $+0.118$ (Figure 1b). The bootstrap interval conditions on every design choice, so it is narrower than the honest spread: across split point, target threshold and cleaning cutoff the second gain ranges $+0.068$ to $+0.130$, and it rises with training volume. Nor does the disclosed field editing explain it: reducing the intake block to Category alone — dropping Impact and Urgency, which are edited, and Priority, which is a deterministic function of them — gives $+0.106$, and restricting to the 44,227 never-edited incidents gives $+0.107$.

It is not an artifact of the estimator. The dimensionality null above rules out the regularisation burden of 2,554 sparse columns, but a reader may reasonably want the effect shown under an estimator where that burden cannot arise at all. We therefore re-represent item identity as a single cross-fitted target-encoded column — fitted on training only, out-of-fold within training, so no row sees its own outcome — which makes the item one continuous feature rather than thousands of indicators, and makes histogram boosting usable on a variable that otherwise collapses 2,554 items into 137 bins. Across one-hot logistic regression, logistic regression on the encoded item, and boosting on the encoded item, the first rung ranges $+0.173$ to $+0.183$, the second $+0.092$ to $+0.103$, and the shrinkage 43% to 47%. Encoding a *shuffled* item column under the same pipeline returns -0.0001 ± 0.0016 and $+0.0042 \pm 0.0020$, so the encoder is not leaking; we report the second figure rather than round it away, and it bounds how much of the boosting rung could be encoding artifact.

Quantity	AUC
item's gain over intake, $A_i - A_0$	$+0.183$
item's gain once the queue is present, $A_{qi} - A_q$	$+0.103$
queue's gain over intake, $A_q - A_0$	$+0.082$
queue's gain once the item is present, $A_{qi} - A_i$	$\mathbf{+0.002}$

Table 2: The overlap, read four ways. The final row is the one that matters.

5 Where the Difference Goes

The two rows differ by 0.080. That number is not itself informative — for any two variables, the drop in one's measured gain when the other joins the baseline equals the drop in the other's, so it restates the table rather than explaining it. Two direct measurements do explain it.

First: adding the routing queue to a model that already knows the affected item is worth $+0.0017$ $[+0.0001, +0.0034]$. It is positive, and it clears a matched-dimension null of -0.0009 ± 0.0006 that no draw reaches — so it is not the cost of adding columns. But across split points and penalties it ranges $+0.0002$ to $+0.0072$, so the

honest statement is that the queue's unique contribution is *under* 0.01 *AUC and not resolvable more finely*. Whatever the queue contributes, the item column has almost entirely absorbed it.

Second, and more directly: the queue's predictive content is nearly a function of the item. Randomising the queue label *within* each item — which destroys the queue's own identity while preserving what the item implies about it — still retains 91% of the queue's +0.082 gain. The matched floor, the same randomisation within cells drawn to reproduce the item mass profile, retains 2%. The margin is 89 points.

How far this goes, and where it stops. Two things bound the claim, and both are measurements rather than caveats. Without any estimator, item identity carries 60.4% of the queue's information and the queue carries 19.6% of the item's — the relationship is real and strongly asymmetric, which is why we do not run the argument backwards. But it is not determination: a modal-queue lookup keyed on item identity reproduces the desk's choice on 90.3% of test incidents against 78.6% for always guessing the largest queue, and class-balanced it reaches only 34.1%. So the supported statement is that item identity resolves the coarse routing contrast and much less of the fine one — not, as an earlier draft of this section had it, that the routing decision is nearly a function of the item.

The shrinkage is graded in the queue's resolution. The coarse-versus-fine reading makes a prediction, and it holds. Replacing the queue with progressively coarser versions of itself and re-running the same ladder gives shrinkages of 27%, 28%, 36% and 44% at two, four, eleven and 49 levels (Figure 3a). A single binary flag — did the desk send this to the main pool, or not — already recovers 61% of the queue's baseline gain and 63% of the shrinkage it causes. For a practitioner this matters more than the headline: the field that has to be admitted to the baseline is one bit of routing, far cheaper to reproduce in another organisation than a 49-way taxonomy.

What we do not claim. The reverse framing — that the item column "stands in for" the queue — is not established. We measured it, by randomising items within queues, and obtained 44% of the item's gain; but a random 50-cell grouping of items that knows nothing about routing retains 25%, and a grouping matched on the queue's mass profile retains −7%. Defensible nulls disagree about the floor, so the number is not interpretable and we exclude it. Nor do we claim a direction of causation. The figures above are equally consistent with the desk reading the item off the ticket and routing accordingly, and with the queue being a separate signal that happens to align; the log records no counterfactual routing, so the question is not identified here. Our claim is about what an analyst's baseline choice does to a measured number, which does not depend on which is true.

6 What the Difference Buys

An AUC difference is not a quantity a service desk can act on, so we state the same comparison as a detection problem. Fix a review capacity — the share of arrivals a desk can afford to look at before routing — rank the test incidents by predicted risk, and count how many reassignment-bound incidents each model surfaces.

	caught			
Review capacity	base	+ item	extra	per month
5% (682)	597	653	+56	30
10% (1,364)	1,119	1,153	+34	18
20% (2,727)	1,823	1,857	+34	18

Table 3: Reassignment-bound incidents surfaced in the 13,637-incident test window at a fixed review capacity, with the queue in the baseline. Omitting the queue would credit item identity with 303, 432 and 501 instead.

At 10% capacity, knowing the affected item surfaces 34 more of them than the intake-plus-queue baseline does, or 18 a month at this log's volume, moving precision from 82.0% to 84.5% against a 37.2% base rate (Table 3). Had the queue been left out of the baseline, the same capacity would have credited item identity with 432 extra catches. **The omission overstates the operational gain by a factor of** 12.7, and by 5.4 to 14.7 across the three capacities.

That factor is much larger than the ratio of the two AUC gains, which is 1.8, and the discrepancy is itself the practical point. AUC averages over every operating point; a capacity-limited desk works at one, near the top of the ranking, where the queue-aware baseline is already close to its ceiling and the item column has far less left to add. A business case that reports an AUC delta understates how much of its own headline is the baseline choice. What this does *not* say is that a surfaced incident is a corrected one: the log records no intervention, so the recoverable share is not identified here.

Three target definitions. Because reassignment is common, we repeat the ladder at stricter thresholds. Requiring two or more reassignments (21.8% of incidents) gives +0.131 and +0.068; three or more (10.6%) gives +0.151 and +0.080. The magnitude is threshold-dependent and not monotone, but every interval excludes zero, the ordering never reverses, and the shrinkage stays between 44% and 48%. The transferable quantity is the shrinkage, not the gain.

7 Does It Survive a Change of Task?

A gap that appears on one target may be a property of that target. We repeated the ladder on two further outcomes from the same cohort, holding the cohort, the split, the estimator, the intake block and the null construction fixed, and changing only what is predicted.

Reopening — whether an incident was reopened after resolution — is close to an independent failure mode: it fires on 2,096 incidents and correlates with reassignment at +0.14. *Long handling* — handle time above the training third quartile — correlates at +0.40, so it partly re-measures the primary target, and we report it as corroboration rather than as independent evidence.

The shrinkage reproduces on both. On reopening, item identity is worth +0.083 against the intake block and +0.055

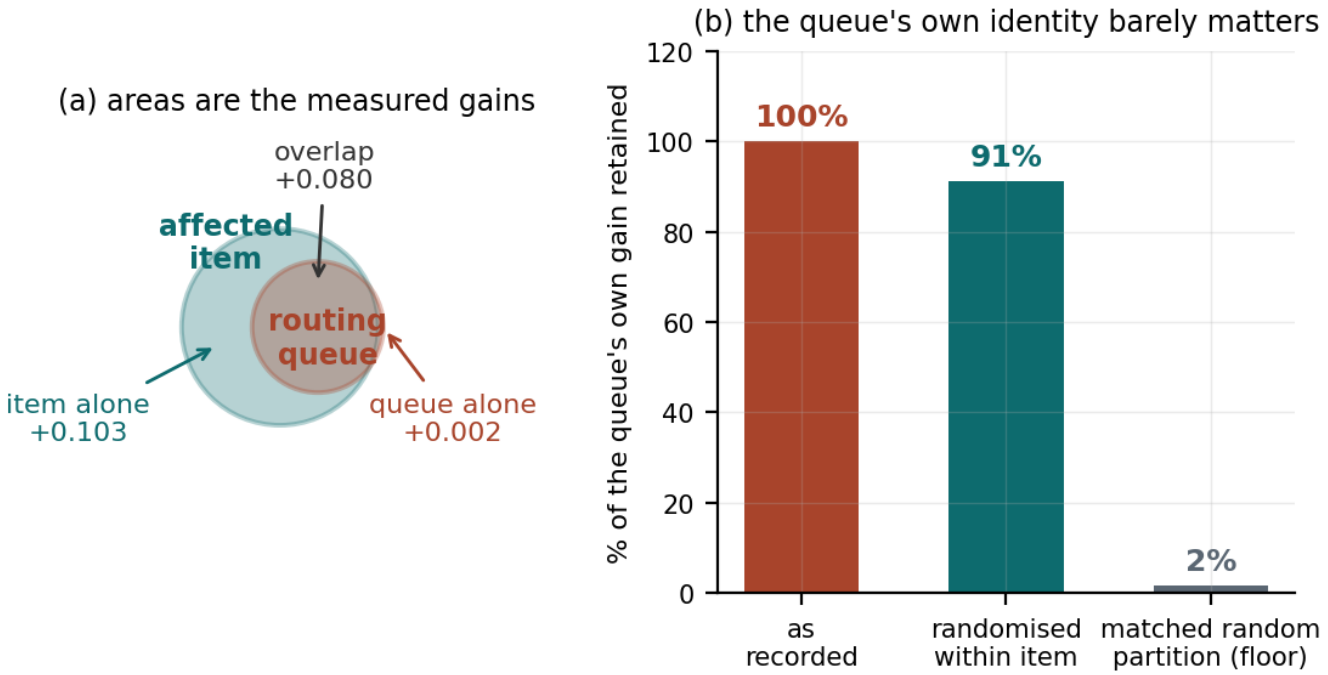

Figure 2: (a) The overlap, drawn to scale: each circle's area is that field's measured gain over the intake block, and the centre distance is solved so that the intersection equals the measured overlap. The queue lies almost entirely inside the item. (b) Randomising the queue label within each item destroys the queue's own identity and costs little; the same randomisation within cells drawn to reproduce the item mass profile destroys almost everything.

once the queue is admitted, a reduction of 33%. On long handling the two figures are $+0.118$ and $+0.078$, a reduction of 34%. Every rung on every target sits outside its matched-dimension null, and across five split points the reduction ranges from 30% to 46% over the three targets.

Two qualifications. Reopening is rare and its intervals are correspondingly wide — its two gains stand 5.5 and 4.2 pooled standard deviations from their nulls, far short of the margins in Section 4 — so it establishes the direction of the effect and not its size. And the reduction is not constant across targets: it is largest on the one the routing queue is most directly about. That is what the mechanism in Section 5 predicts, and it is why we state the transferable claim as the existence of the gap rather than its magnitude.

8 One Thing We Could Not Establish

A third field, the knowledge-article reference, sits on the same `Open` row, is 100% populated, and costs nothing. Adding it to the baseline raises AUC to 0.805 and drives the measured value of item identity to -0.003. Whether that figure belongs in Table 1 depends on whether the field is available at creation, and we could not determine this.

We tried two structural tests and report both failures. A field that varies within an incident is certainly a per-event observation; the knowledge reference never varies. But neither does `Interaction ID`, which has 45,426 distinct values for 45,455 incidents and is plainly a creation-time key — so constancy is evidence of granularity, not of timing. We then tried exact identity with the closed record: the knowledge reference matches its closed-record column for 100.000000% of incidents. But restricted to the 42,151 incidents with exactly one related interaction, `Interaction ID` matches its own closed-record column for 99.997628% — one mismatch — against a pass mark of 100.000000%. The second test cannot separate its own counterexample either.

We therefore make no claim about this field. The -0.003 is additionally inside the dimensionality null described in Section 4, and changes sign under per-condition penalty tuning and at other split points, so it is not resolvable here on either ground. This section can be read as the paper's own thesis turned on itself, and we think that reading is correct: our headline is contingent on a field-admission decision we cannot adjudicate, which is precisely the exposure we are arguing every CMDB business case carries unstated. We report the attempt because the negative is the transferable part: a field being free, populated and present on an opening record does not make it creation-time, and we know of no test on this export that settles it.

9 Estate Concentration

Required CMDB scope is bounded by a property of the estate that needs no model. Here the top 8 items generate 30.2% of incidents, the top 64 generate 70.5%, and the top 128 — 5.0% of the training vocabulary — generate 82.0%. An organisation can compute this from a frequency count before funding anything.

Identifying only those items recovers most of the measured value. Averaged over five temporal split points, the top 8

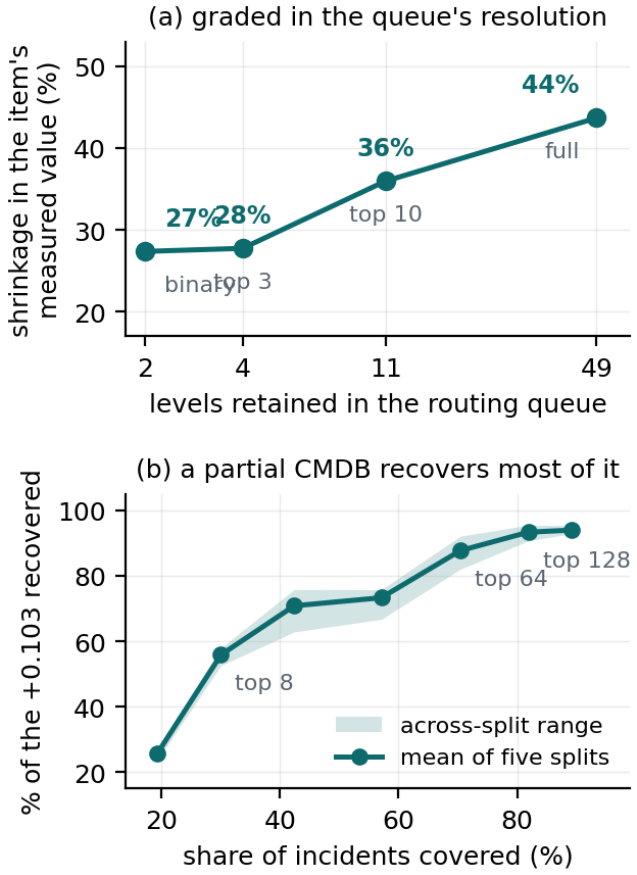

Figure 3: (a) The shrinkage is graded in the queue's resolution. (b) How much of the item column's contribution a partial CMDB recovers, averaged over five temporal split points; the band is the across-split range.

recover 56% [53, 58] of the +0.103, the top 64 recover 88% [82, 92], and the top 128 recover 93% [91, 95] (Figure 3b). We report the mean and range rather than one split's curve because a single curve is not monotone at this resolution: at $k = 32$ the across-split spread is 9 points, so adjacent values of k cannot be read against each other. The same holds with the queue removed from the baseline — 89% at $k = 64$ — so this is a property of the estate's concentration and not of our field-admission decision. We make no claim about how much of the identity gain an arbitrary coverage level recovers; an earlier draft did, from a comparison in which one of three selection rules had no observations at the coverage levels where the claim was made.

10 Limitations

One organisation — of necessity, not choice: no other public ITSM log records the affected item (Section ef-sec:background) — one platform, one task, six months. The data are from 2013 and 2014 and from a single on-premises ITSM product; estate composition, routing practice and tooling have all moved since, and we make no claim that the magnitudes transfer to a cloud-native estate. The values +0.183 and +0.103 are properties of this estate; the transferable claims are that the gap exists, that it is large, that it is graded in the free field's resolution, and that the mechanism is the item column proxying for the queue. Reassignment is a proxy for misrouting and also fires on legitimate escalation. The within-queue shuffle preserves queue membership exactly but not any other correlate of item identity, so it bounds the proxy share rather than isolating it perfectly. The routing queue's distribution drifts substantially between our training and test halves, which is realistic for a temporal split but means the queue-aware baseline is being asked to generalise across a shift.

11 Conclusion

Knowing which configuration item an incident concerns is worth +0.183 or +0.103 AUC for predicting reassignment, depending on whether the baseline includes a routing queue the service desk already records — and stated as a detection problem at a fixed review capacity, the same choice moves the credited benefit by more than a factor of ten. The difference is an overlap: the queue's predictive content is very nearly carried by the item, so a model given the item has already been told most of what the queue would say. A business case built on the first figure is not measuring a CMDB. It is measuring a CMDB plus a decision the organisation had already made for free.

Use of AI Systems

An AI assistant (Anthropic's Claude) was used in preparing this work: to implement and refactor analysis code, to construct adversarial challenges to intermediate claims — several of the withdrawals recorded above originated that way — and to draft and edit prose that the author reviewed, revised and verified. All analyses were specified, run and checked by the author, who takes full responsibility for the content, including every numeric claim and every reference. No text was included that the author has not read and confirmed. The repository accompanying this paper contains the working notes from that process.

Acknowledgements

This work benefited substantially from the review and domain challenge of a practitioner working in IT service management. Several of the analyses withdrawn from earlier versions were withdrawn because of objections raised in that review.

Reproducibility

The dataset is public (van Dongen 2014). Analysis and figure code, and a checker that compares each quantity here against a generated result file, are available at https://github.com/sudhirdixit1/cmdb-routing-queue-baseline.

References

Amaral, C.; Fantinato, M.; and Peres, S. M. 2018. Incident Management Process Enriched Event Log. UCI Machine Learning Repository. Extracted from the audit system of a

ServiceNow instance; 141,712 events over 24,918 incidents. CC BY 4.0.

Bouthillier, X.; Delaunay, P.; Bronzi, M.; Trofimov, A.; Nichyporuk, B.; Szeto, J.; Mohammadi Sepahvand, N.; Raff, E.; Madan, K.; Voleti, V.; Ebrahimi Kahou, S.; Michalski, V.; Arbel, T.; Pal, C.; Varoquaux, G.; and Vincent, P. 2021. Accounting for Variance in Machine Learning Benchmarks. In *Proceedings of Machine Learning and Systems (MLSys)*.

Chen, J.; He, X.; Lin, Q.; Zhang, H.; Hao, D.; Gao, F.; Xu, Z.; Dang, Y.; and Zhang, D. 2019. Continuous Incident Triage for Large-Scale Online Service Systems. In *Proceedings of the 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*.

Covert, I.; Lundberg, S.; and Lee, S.-I. 2020. Understanding Global Feature Contributions With Additive Importance Measures. In *Advances in Neural Information Processing Systems 33 (NeurIPS)*.

Ferrari Dacrema, M.; Cremonesi, P.; and Jannach, D. 2019. Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches. In *Proceedings of the 13th ACM Conference on Recommender Systems (RecSys)*.

Gundersen, O. E.; and Kjensmo, S. 2018. State of the Art: Reproducibility in Artificial Intelligence. In *Proceedings of the 32nd AAAI Conference on Artificial Intelligence*.

Kapoor, S.; and Narayanan, A. 2023. Leakage and the Reproducibility Crisis in Machine-Learning-Based Science. *Patterns*, 4(9).

Kaufman, S.; Rosset, S.; Perlich, C.; and Stitelman, O. 2012. Leakage in Data Mining: Formulation, Detection, and Avoidance. *ACM Transactions on Knowledge Discovery from Data*, 6(4).

Lin, Q.; Hsieh, K.; Dang, Y.; Zhang, H.; Sui, K.; Xu, Y.; Lou, J.-G.; Li, C.; Wu, Y.; Yao, R.; Chintalapati, M.; and Zhang, D. 2018. Predicting Node Failure in Cloud Service Systems. In *Proceedings of the 26th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*.

Marrone, M.; and Kolbe, L. M. 2011. Impact of IT Service Management Frameworks on the IT Organization: An Empirical Study on Benefits, Challenges and Processes. *Business & Information Systems Engineering*, 3(1): 5–18.

Notaro, P.; Cardoso, J.; and Gerndt, M. 2021. A Survey of AIOps Methods for Failure Management. *ACM Transactions on Intelligent Systems and Technology*, 12(6).

Remil, Y.; Bendimerad, A.; Mathonat, R.; and Kaytoue, M. 2024. AIOps Solutions for Incident Management: Technical Guidelines and A Comprehensive Literature Review. *arXiv preprint arXiv:2404.01363*.

Rendle, S.; Zhang, L.; and Koren, Y. 2019. On the Difficulty of Evaluating Baselines: A Study on Recommender Systems. *arXiv preprint arXiv:1905.01395*.

Steeman, W. 2013. BPI Challenge 2013, Incidents. 4TU.ResearchData. Volvo IT incident and problem management, VINST system.

Suriadi, S.; Andrews, R.; ter Hofstede, A. H. M.; and Wynn, M. T. 2017. Event Log Imperfection Patterns for Process Mining: Towards a Systematic Approach to Cleaning Event Logs. *Information Systems*, 64: 132–150.

Teinemaa, I.; Dumas, M.; La Rosa, M.; and Maggi, F. M. 2019. Outcome-Oriented Predictive Process Monitoring: Review and Benchmark. *ACM Transactions on Knowledge Discovery from Data*, 13(2): 17:1–17:57.

van der Aalst, W. M. P. 2016. *Process Mining: Data Science in Action*. Springer, 2nd edition.

van Dongen, B. F. 2014. BPI Challenge 2014. 4TU.ResearchData. Rabobank Group ICT, HP Service Manager; incident, change and interaction details.

Verenich, I.; Dumas, M.; La Rosa, M.; Maggi, F. M.; and Teinemaa, I. 2019. Survey and Cross-Benchmark Comparison of Remaining Time Prediction Methods in Business Process Monitoring. *ACM Transactions on Intelligent Systems and Technology*.

Weytjens, H.; and De Weerdt, J. 2022. Creating Unbiased Public Benchmark Datasets with Data Leakage Prevention for Predictive Process Monitoring. In *Business Process Management Workshops*, volume 436 of *Lecture Notes in Business Information Processing*, 18–29. Springer.

Williamson, B. D.; Gilbert, P. B.; Carone, M.; and Simon, N. 2021. Nonparametric Variable Importance Assessment Using Machine Learning Techniques. *Biometrics*, 77(1): 9–22.

Zhang, L.; Jia, T.; Jia, M.; Wu, Y.; Liu, A.; Yang, Y.; Wu, Z.; Hu, X.; Yu, P. S.; and Li, Y. 2024. A Survey of AIOps for Failure Management in the Era of Large Language Models. *arXiv preprint arXiv:2406.11213*.